

LEVEL 2

Chapter 3: Capacity Estimation

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 3: Capacity Estimation ("Back-of-the-Envelope Math")

← [Chapter 2: Fundamentals](#) · [Index](#) · Next → [Chapter 4: Core Building Blocks](#)

3.1 Why Interviewers Ask You to Do Math on a Whiteboard

This step isn't about getting a precise number — nobody expects you to be within 5%. It exists to test whether your architecture decisions are **grounded in scale** rather than vibes. "I'll use Redis for caching" means something different at 100 RPS versus 500,000 RPS. Skipping estimation is one of the most commonly cited mistakes in real interview feedback (see Chapter 23) because it means every subsequent decision you make is unanchored — the interviewer can't tell if you actually understand *why* you need a queue, or if you just know queues are a thing system designs have.

The one habit that matters more than the math itself: round aggressively, state your assumptions out loud, and move on. 1 million $\approx 10^6$. A day has $\sim 86,400$ seconds $\approx 10^5$ for quick mental math (86,400 rounds to "about 100,000" for rough RPS conversions — always say you're rounding). Nobody has ever lost interview points for rounding 86,400 to 100,000; people lose points for spending four silent minutes doing long division.

3.2 The Reusable Framework

Memorize this sequence — it's the same five moves for every problem:

1. **Users → DAU/MAU.** Get (or assume, out loud) daily active users.
2. **DAU → requests/day → average RPS.** $\text{Average RPS} = (\text{DAU} \times \text{actions per user per day}) / 86,400$.
3. **Average RPS → Peak RPS.** Traffic isn't flat. Apply a **peak multiplier**, typically **2x-3x** average for consumer apps (higher — 5x-10x — for flash-sale/event-driven traffic like Black Friday or IPL streaming).
4. **Storage.** $\text{Storage/day} = \text{actions/day} \times \text{size per record}$, then multiply by retention period and **replication factor** (typically 3x for durability).
5. **Bandwidth.** $\text{Bandwidth} = \text{requests/sec} \times \text{average payload size}$. Do this for both ingress (uploads) and egress (downloads) separately — they're often very different.

Reference numbers worth memorizing

Quantity	Value
Seconds in a day	86,400 ($\approx 10^5$)
Seconds in a month	~ 2.6 million ($\approx 2.6 \times 10^6$)
1 KB	10^3 bytes
1 MB	10^6 bytes
1 GB	10^9 bytes
1 TB	10^{12} bytes
Typical read:write ratio, consumer social apps	100:1 to 1000:1
Typical read:write ratio, e-commerce	10:1 to 100:1
Typical peak multiplier (normal day)	2 $\times$ -3 $\times$ average
Typical peak multiplier (flash sale/viral event)	5 $\times$ -10 $\times$ + average
Replication factor (durability)	3 (standard for distributed storage — Kafka, Cassandra, HDFS all default here)

Latency budget cheat sheet (useful when asked "why is this slow")

Operation	Approximate latency
L1/L2 cache reference	~ 1 nanosecond
Main memory (RAM) reference	~ 100 nanoseconds
SSD random read	~ 100 microseconds
Round trip within same data center	~ 0.5 millisecond
Redis GET (same region)	~ 1 millisecond
Read 1MB sequentially from SSD	~ 1 millisecond
Round trip, cross-region (e.g., Mumbai $\leftrightarrow$ Virginia)	~ 150 - 250 milliseconds
Disk seek (spinning disk, rare now)	~ 10 milliseconds

(You don't need exact figures — you need the relative ordering: memory beats SSD beats network-same-DC beats network-cross-region, by roughly 2-3 orders of magnitude at each step. That ordering alone explains why caching works and why multi-region architectures are hard.)

3.3 Ten Fully Worked Examples

Each example is deliberately terse — this is what you'd actually say/write in 5–7 minutes on a whiteboard, not a essay.

Example 1 — URL Shortener

Assumptions: 100M new URLs shortened per month. 100:1 read:write ratio (people click short links far more than they create them).

- Writes/sec (avg) = $100\text{M} / 2.6\text{M sec} \approx \sim\mathbf{40\ writes/sec}$
- Reads/sec (avg) = $40 \times 100 \approx \sim\mathbf{4,000\ reads/sec}$
- Peak reads ($\times 3$) $\approx \sim\mathbf{12,000\ reads/sec}$
- Storage per URL: ~ 500 bytes (long URL + short code + metadata). $100\text{M/month} \times 12\text{ months} \times 5\text{ years} \times 500\text{ bytes} \approx \sim\mathbf{3\ TB\ over\ 5\ years}$ (before replication) $\rightarrow \sim 9\text{ TB}$ with 3x replication.
- **Conclusion this math drives:** Read load is cache-dominated — this screams "put a cache (Redis) in front of the DB, since reads outnumber writes 100:1 and most reads hit a small set of popular links (Zipfian distribution)." Storage is small enough that a single well-indexed relational DB or key-value store easily handles it without sharding for years.

Example 2 — WhatsApp-style Messaging

Assumptions: 500M DAU, 40 messages sent per user per day average.

- Messages/day = $500\text{M} \times 40 = 20\text{B messages/day}$
- Avg messages/sec = $20\text{B} / 86,400 \approx \sim\mathbf{230,000\ msgs/sec}$
- Peak ($\times 3$) $\approx \sim\mathbf{700,000\ msgs/sec}$
- Storage per message: ~ 100 bytes (text) average. $20\text{B} \times 100\text{ bytes/day} \approx \mathbf{2\ TB/day}$ raw, $\sim 6\text{ TB/day}$ with 3x replication. Over a year (assuming most old messages get archived to cold storage, not kept hot): tens of PB in cold storage.
- **Conclusion:** This volume rules out a single relational database outright — you need horizontally partitioned storage (partition by conversation ID or user ID), a message queue/log (Kafka-like) to absorb write bursts and decouple senders from the fan-out/delivery pipeline, and a clear hot/cold storage split (recent messages in fast storage, old ones archived).

Example 3 — Instagram-style Feed

Assumptions: 200M DAU, each user checks feed 5x/day (reads), 1 post per 10 users/day (writes, i.e., 10% of DAU posts daily).

- Feed reads/day = $200M \times 5 = 1B$ reads/day $\rightarrow$ avg $\approx \sim 11,600$ reads/sec, peak ($\times 3$) $\approx \sim 35,000$ reads/sec
- Posts/day = 20M $\rightarrow$ avg $\approx \sim 230$ writes/sec, peak $\approx \sim 700$ writes/sec
- Read:write ratio $\approx 50:1$ — read-heavy.
- Media storage: assume 20M posts/day $\times$ 2MB avg (photo+thumbnails) = **40 TB/day** of media $\rightarrow$ this alone tells you object storage (S3-class), not a database, is the only sane place for the media itself; the DB only stores metadata + pointers.
- **Conclusion:** Classic fan-out problem (Chapter 15/16 covers this in full) — do you push new posts to every follower's feed cache at write time ("fan-out on write," fast reads, expensive for celebrities with 100M followers) or compute the feed at read time by pulling from people you follow ("fan-out on read," cheap writes, slower reads)? At this scale the real answer is hybrid: fan-out on write for normal users, fan-out on read (or a separate path) for celebrity accounts — exactly the trade-off Instagram and Twitter have both spoken about publicly.

Example 4 — Food Delivery (Swiggy/Zomato/Talabat-style)

Assumptions: 10M DAU, 0.3 orders/user/day (30% of DAU orders), each order generates ~ 15 location update pings from the delivery partner over its lifetime.

- Orders/day = 3M $\rightarrow$ avg $\approx \sim 35$ orders/sec, peak (lunch/dinner rushes, $\times 5$) $\approx \sim 175$ orders/sec
- Location pings/day = $3M \times 15 = 45M$ $\rightarrow$ avg $\approx \sim 520$ pings/sec, peak $\approx \sim 2,600$ pings/sec
- Storage: order records are small ($\sim 2KB$ each with items/pricing) — $3M \times 2KB = 6GB$ /day, trivial. Location pings are the real volume driver if kept long-term, but typically only the *latest* location per active order needs to be hot (Redis), with history archived or discarded.
- **Conclusion:** Two very different sub-systems with different NFRs living under one product — the *order/payment* path needs strong consistency and durability (Chapter 20's ledger patterns apply), while the *location ping* path is high-volume, loss-tolerant, latest-value-wins (a job for Redis/a time-series store, not the primary transactional DB). Conflating these two into one data store is a common beginner mistake.

Example 5 — E-commerce (Flipkart/Amazon-style)

Assumptions: 50M DAU browsing, 1% conversion → 500K orders/day. Product catalog: 100M SKUs.

- Browse/search requests: $50M \times 10 \text{ page views/day} = 500M/\text{day} \rightarrow \text{avg} \approx \sim 5,800 \text{ RPS}$, peak (flash sale, $\times 8$) $\approx \sim 46,000 \text{ RPS}$
- Orders/day = 500K $\rightarrow \text{avg} \approx \sim 6 \text{ orders/sec}$, peak (sale event, $\times 10$) $\approx \sim 60 \text{ orders/sec}$ — note orders are tiny in RPS terms compared to browsing; browsing/search is the scaling problem, not checkout.
- Catalog storage: $100M \text{ SKUs} \times \sim 5KB \text{ (structured data + attributes)} = 500GB$ — very manageable for a well-indexed DB; images/video go to object storage separately and are the real bulk (potentially $100M \times 2MB = 200TB$).
- **Conclusion:** Read-heavy browsing traffic (CDN + cache + search index like Elasticsearch) is architecturally a completely different problem from the low-volume-but-must-not-fail checkout/payment path (Chapter 20). Flash sales are the real NFR-defining event — design for the $\times 8$ - 10 peak multiplier explicitly, with inventory as the classic hot-key/race-condition problem (Chapter 4's cache section, "hot keys").

Example 6 — Ride-Sharing (Uber/Careem-style)

Assumptions: 5M daily riders, 1M active drivers, each active driver sends a location ping every 4 seconds during a ~ 6 -hour active window.

- Location pings/sec = $1M \text{ drivers} \times (1 \text{ ping} / 4 \text{ sec}) \approx \sim 250,000 \text{ pings/sec}$ sustained during active hours — this dwarfs the ride-request traffic itself.
- Ride requests/day: assume 5M rides/day $\rightarrow \text{avg} \approx \sim 58 \text{ requests/sec}$, peak (rush hour, $\times 4$) $\approx \sim 230 \text{ requests/sec}$.
- **Conclusion:** The location-ping ingestion pipeline, not the ride-matching logic itself, is the dominant scaling challenge in terms of raw throughput — it needs a high-throughput ingestion layer (Kafka-class) feeding a geospatial index (geohash/S2-cell-based, held largely in memory/Redis) for the driver-matching service to query with sub-second latency. This is exactly the shape of Uber's and Careem's own published architecture discussions.

Example 7 — Payment System (Razorpay/PhonePe/Stripe-style)

Assumptions: 20M transactions/day across a mid-large PSP.

- Avg TPS = $20M / 86,400 \approx \sim 230 \text{ TPS}$, peak (salary day / festival sale, $\times 5$) $\approx \sim 1,150 \text{ TPS}$. (For comparison, Visa's network is engineered for $\sim 65,000 \text{ TPS}$ peak capacity globally — useful to know the ceiling you're nowhere near at this scale, but the *design principles* — idempotency, strong consistency on the ledger — matter regardless of absolute scale.)

- Storage: each transaction record with full audit trail $\approx 2\text{KB}$. $20\text{M} \times 2\text{KB/day} \approx 40\text{GB/day}$ — small in raw bytes, but this data has an extremely strict **durability and immutability** requirement (append-only ledger, Chapter 20) that outweighs raw volume as the design driver.
- **Conclusion:** This is a case where the numbers are almost secondary to the NFRs — the design is driven by idempotency (never double-charge on retry), strong consistency (the ledger cannot be eventually consistent), and auditability, not by raw throughput. Say this explicitly in an interview — it shows you know capacity estimation isn't the *only* lens.

Example 8 — Video Streaming (Netflix/YouTube-style)

Assumptions: 50M DAU, average watch time 90 min/day, average bitrate 5 Mbps (mixed SD/HD/4K).

- Concurrent streams at peak (assume 15% of DAU watching simultaneously during peak evening hours) = 7.5M concurrent streams.
- Peak bandwidth = $7.5\text{M} \times 5\text{Mbps} \approx \mathbf{37.5\text{ Tbps}}$ — a genuinely enormous number, and the whole reason this is a CDN-first problem: no origin infrastructure serves this directly; you push content to edge caches (own CDN like Netflix's Open Connect, or a commercial CDN like CloudFront/Akamai) sitting physically inside or near ISPs.
- Storage: original + multiple transcoded bitrate/resolution variants per video. A 2-hour movie might have 10+ variants (adaptive bitrate streaming) totaling $\sim 30\text{GB}$ across all variants; catalog of 20,000 titles $\approx 600\text{TB}$ — again, object storage + CDN, not a database.
- **Conclusion:** Bandwidth, not requests/sec, is the binding constraint, which flips the whole architecture toward CDN edge delivery, adaptive bitrate streaming (multiple pre-encoded qualities, client picks based on network conditions), and asynchronous transcoding pipelines — none of which resemble a typical CRUD-app design.

Example 9 — Notification System

Assumptions: 100M users, average 5 notifications/user/day across push/SMS/email.

- Notifications/day = 500M $\rightarrow$ avg $\approx \mathbf{\sim 5,800/\text{sec}}$, peak (broadcast campaign to all users at once, e.g., a sale announcement) can spike to **millions within a few minutes** — this "thundering herd" broadcast case, not steady-state average, is what actually breaks naive designs.
- **Conclusion:** The design must handle bursty fan-out (a single trigger $\rightarrow$ millions of individual sends) via a queue-backed worker pool that can scale out horizontally and rate-limit against each downstream provider's (APNs, FCM, Twilio, SES) own throughput limits — this is fundamentally a queueing/backpressure problem, not a database problem (Chapter 6/7).

Example 10 — File Storage (Dropbox/Google Drive-style)

Assumptions: 50M users, average 5GB stored/user, 2% of users upload a new file on a given day averaging 10MB.

- Total storage = $50M \times 5GB = \mathbf{250\ PB}$ — object storage at this scale is non-negotiable; no filesystem or single-server solution applies.
- Upload writes/day = $1M\ users \times 10MB = 10TB/day$ ingress $\rightarrow avg \approx \mathbf{\sim 115\ MB/sec}$ sustained ingress, bursty around business hours.
- **Conclusion:** This is a metadata + blob-storage split: a database (or metadata service) tracks file ownership, versions, permissions, and sharing, while the actual bytes live in object storage (chunked for large files, deduplicated where possible, as Dropbox has published about their own "block-level sync" approach). The interesting HLD problems are metadata consistency (rename/move operations across a hierarchy) and sync/conflict resolution across devices, not raw storage — the storage layer is "use S3-class storage and move on."

3.4 What This Chapter Should Leave You With

Notice the pattern across all ten: **the arithmetic itself is almost never the point.** The point is that in each case, the numbers *justified* a specific architectural fork — cache-in-front-of-DB, hybrid fan-out, separate hot/cold paths, CDN-first, queue-backed fan-out, metadata/blob split. That's the muscle you're building: use the math to *earn* your architecture decisions instead of asserting them. Chapter 13's Universal Interview Framework builds this estimation step into Step 2 of the ten-step sequence — you'll do a lighter version of this exercise in every single mock interview from Chapter 28 onward.

Next $\rightarrow$ [Chapter 4: Core Building Blocks](#) — load balancers, CDNs, reverse proxies, API gateways, caching, and how to choose between SQL and NoSQL.